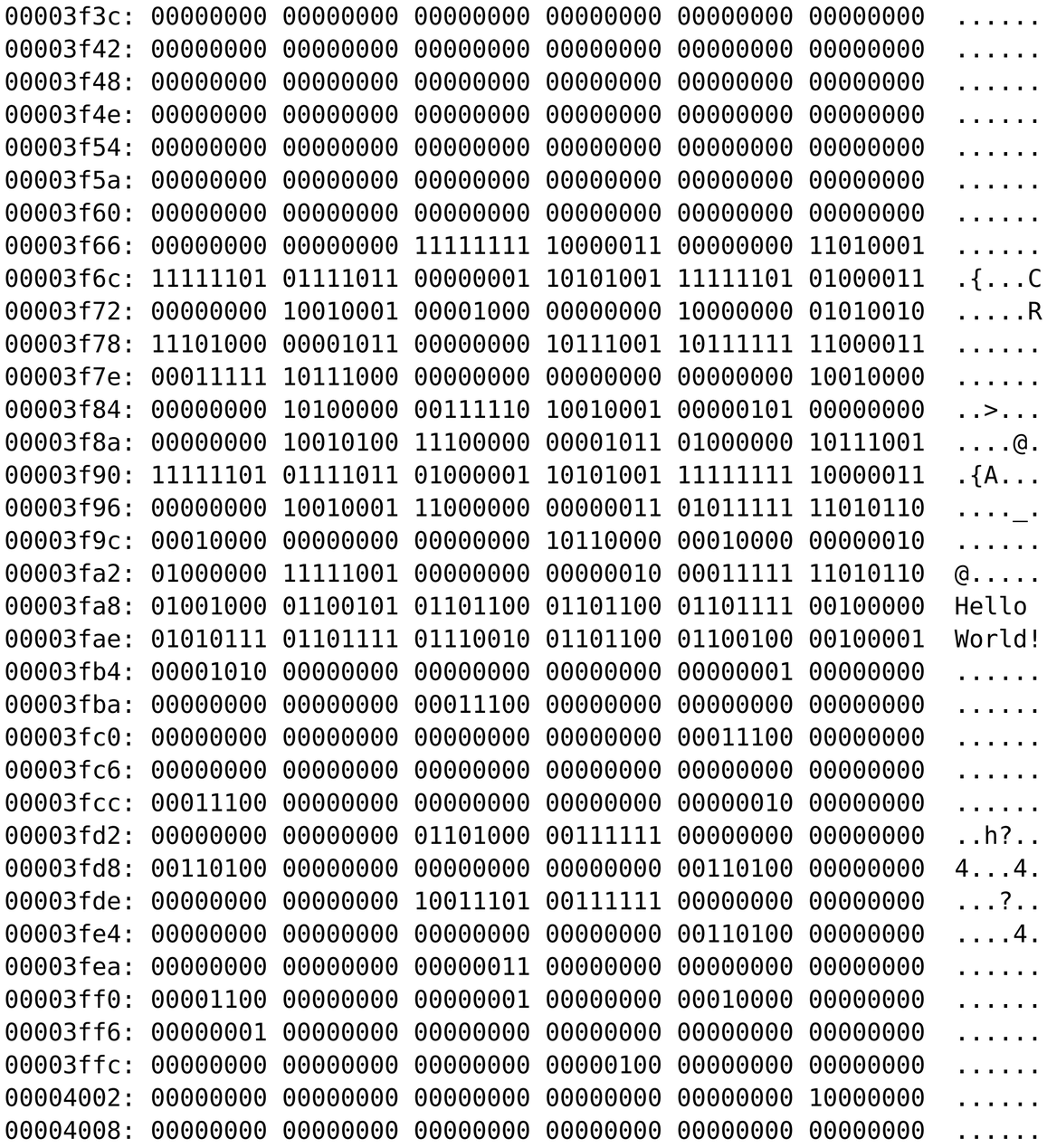


Storing and Manipulating Data

Alex Breuer

Overview and Schedule

- Use computers to store data, retrieve data, and manipulate data
- Our data can be diverse:
  - Numerical: integers and floats
  - Textual: characters and strings
  - Programs
- Challenge: computers only store bits (zeroes and ones)
- Now: integers, chars and strings
- Later: programs and floats



Number Systems

Decimal Number System

- Symbols: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9
- Base 10:

$$7743_{10} = 7 \cdot 10^3 + 7 \cdot 10^2 + 4 \cdot 10^1 + 3 \cdot 10^0$$

- Positional: $7743_{10} \neq 4377_{10}$
- Typically assumed when writing a number, i.e. writing “7743” means 7743_{10}

Base		
16	**10**	**2**
0	0	0
1	1	1
2	2	10
3	3	11
4	4	100
5	5	101
6	6	110
7	7	111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111
10	16	10000
11	17	10001
12	18	10010

Binary Number System

- Symbols: 0, 1
- Base 2:

  Most significant bit ↙ ↙ Least significant bit

  $$100101_2 = 1 \cdot 2^5 + 0 \cdot 2^4 + 0 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0$$
- Positional: $1001_2 \neq 1100_2$
- Definition:
  - Bit: single binary digit
  - Byte: eight bits
- `0b`-prefix used in C/C++, e.g., `0b1001` for 1001_2

Base		
16	**10**	**2**
0	0	0
1	1	1
2	2	10
3	3	11
4	4	100
5	5	101
6	6	110
7	7	111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111
10	16	10000
11	17	10001
12	18	10010

Hexadecimal Number System

- Symbols: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F
- Base 16:
  $\text{B3F0}_{16} = 11 \cdot 16^3 + 3 \cdot 16^2 + 15 \cdot 16^1 + 0 \cdot 16^0$
- Positional: $\text{B3F0}_{16} \neq \text{F0B3}_{16}$
- `0x`-prefix used in C/C++, e.g., `0xB3F0` for B3F0_{16}

Base		
16	**10**	**2**
0	0	0
1	1	1
2	2	10
3	3	11
4	4	100
5	5	101
6	6	110
7	7	111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111
10	16	10000
11	17	10001
12	18	10010

Base b to Base 10: Expand using Positional Notation

Base 7 to base 10:

$$\begin{aligned} 4160_7 &= 4 \cdot 7^3 + 1 \cdot 7^2 + 6 \cdot 7^1 + 0 \cdot 7^0 \\ &= 1372 + 49 + 42 + 0 \\ &= 1463_{10} \end{aligned}$$

Base 16 to base 10:

$$\begin{aligned} 7743_{16} &= 7 \cdot 16^3 + 7 \cdot 16^2 + 4 \cdot 16^1 + 3 \cdot 16^0 \\ &= 28672 + 1792 + 64 + 3 \\ &= 30531_{10} \end{aligned}$$

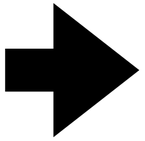
Base 2 to base 10:

$$\begin{aligned} 101001_2 &= 1 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 0 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 \\ &= 32 + 0 + 8 + 0 + 0 + 1 \\ &= 41_{10} \end{aligned}$$

Base 10 to Base b: Repeated Division by b with Remainder

Base 10 to base 16:

$$7743 = 483 \cdot 16^1 + 15 \cdot 16^0$$
$$483 \cdot 16^1 = 30 \cdot 16^2 + 3 \cdot 16^1$$
$$30 \cdot 16^2 = 1 \cdot 16^3 + 14 \cdot 16^2$$
$$1 \cdot 16^3 = 0 \cdot 16^4 + 1 \cdot 16^3$$



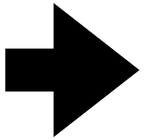
$$7743_{10} = 1E3F_{16}$$

Base		
16	**10**	**2**
0	0	0
1	1	1
2	2	10
3	3	11
4	4	100
5	5	101
6	6	110
7	7	111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111
10	16	10000
11	17	10001
12	18	10010

Base 10 to Base b: Repeated Division by b with Remainder

Base 10 to base 2:

$$743 = 371 \cdot 2^1 + 1 \cdot 2^0$$
$$371 \cdot 2^1 = 185 \cdot 2^2 + 1 \cdot 2^1$$
$$185 \cdot 2^2 = 92 \cdot 2^3 + 1 \cdot 2^2$$
$$92 \cdot 2^3 = 46 \cdot 2^4 + 0 \cdot 2^3$$
$$46 \cdot 2^4 = 23 \cdot 2^5 + 0 \cdot 2^4$$
$$23 \cdot 2^5 = 11 \cdot 2^6 + 1 \cdot 2^5$$
$$11 \cdot 2^6 = 5 \cdot 2^7 + 1 \cdot 2^6$$
$$5 \cdot 2^7 = 2 \cdot 2^8 + 1 \cdot 2^7$$
$$2 \cdot 2^8 = 1 \cdot 2^9 + 0 \cdot 2^8$$
$$1 \cdot 2^9 = 0 \cdot 2^{10} + 1 \cdot 2^9$$



$$743_{10} = 1011100111_2$$

Base		
16	**10**	**2**
0	0	0
1	1	1
2	2	10
3	3	11
4	4	100
5	5	101
6	6	110
7	7	111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111
10	16	10000
11	17	10001
12	18	10010

Shortcut: Binary-Hexadecimal Conversion

- 4 binary digits correspond to 1 hexadecimal digit: $16^1 = 2^4$

- Conversion can be done simply in groups of size four (padded with zeros if necessary)

Base 2 to base 16:

$$11110110010010 1_2 = \ ?$$

Base 16 to base 2:

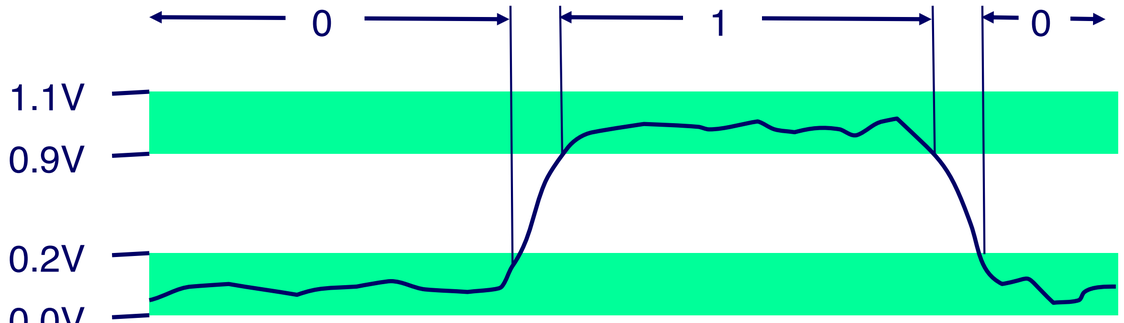
$$\text{B14D}_{16} = \ ?$$

Base		
16	**10**	**2**
0	0	0
1	1	1
2	2	10
3	3	11
4	4	100
5	5	101
6	6	110
7	7	111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111
10	16	10000
11	17	10001
12	18	10010

Representing Integers

Data in Computers

- Computers store information by using binary digits or bits
- Interpretation of bits:
  - Instructions
  - Data (numbers, strings, sets, ...)
- Why use binary logic:
  - Two distinguishable logic levels simplify circuit design
  - Separate voltage ranges for 0 and 1 help tolerate electrical noise



Bryant and O'Hallaron. Computer Systems: A Programmer's Perspective, Third Edition.

Integer Representations: Unsigned

- Assume a sequence of bits in memory, e.g., a byte (eight bits)
- Bits by themselves have no meaning: we have to interpret them
- One interpretation is simple:
  - Bits represent unsigned integer data
  - Decimal value: simply apply base 2 to base 10 conversion
  - n bits represent 0 through $2^n - 1$; eight bits give 0 through 255

Binary	Unsigned	Sign Magnitude	Excess-127	Two's Complement
00000000	0	0	-127	0
00000001	1	1	-126	1
…	…	…	…	…
01111110	126	126	-1	126
01111111	127	127	0	127
10000000	128	-0	1	-128
10000001	129	-1	2	-127
…	…	…	…	…
11111110	254	-126	127	-2
11111111	255	-127	128	-1

Integer Representations: Sign Magnitude

- Most significant bit stores the sign
- Remaining bits store the magnitude
- Arithmetic handles the sign and magnitude separately
- IEEE 754 floating-point formats also use a separate sign bit (later in this course)

Binary	Unsigned	Sign Magnitude	Excess-127	Two's Complement
00000000	0	0	-127	0
00000001	1	1	-126	1
…	…	…	…	…
01111110	126	126	-1	126
01111111	127	127	0	127
10000000	128	-0	1	-128
10000001	129	-1	2	-127
…	…	…	…	…
11111110	254	-126	127	-2
11111111	255	-127	128	-1

Integer Representations: Excess-N

- Unsigned representation with an added bias, typically: $(2^{n-1} - 1)$
- $n := 8$ gives the bias on the right:

$$N = 2^7 - 1 = 127_{10}$$
$$= 1111111_2$$

- Used for exponents of floating point numbers

Binary	Unsigned	Sign Magnitude	Excess-127	Two's Complement
00000000	0	0	-127	0
00000001	1	1	-126	1
…	…	…	…	…
01111110	126	126	-1	126
01111111	127	127	0	127
10000000	128	-0	1	-128
10000001	129	-1	2	-127
…	…	…	…	…
11111110	254	-126	127	-2
11111111	255	-127	128	-1

Integer Representations: Two's Complement

- Most significant bit represents the sign
- Negative numbers are two's complements of their positive counterparts
- Two's complement of an n-bit number y_2:

  $C(y_2) = 2^n - y_2$
- Common method for representing signed numbers

Binary	Unsigned	Sign Magnitude	Excess-127	Two's Complement
00000000	0	0	-127	0
00000001	1	1	-126	1
…	…	…	…	…
01111110	126	126	-1	126
01111111	127	127	0	127
10000000	128	-0	1	$2^7 - 0 = 128$ -128
10000001	129	-1	2	$2^7 - 1 = 127$ -127
…	…	…	…	…
11111110	254	-126	127	$2^7 - 126 = 2$ -2
11111111	255	-127	128	$2^7 - 127 = 1$ -1

Integer Representations: Two's Complement

- Most significant bit represents the sign
- Negative numbers are two's complements of their positive counterparts
- Two's complement of an n-bit number y_2:

  $C(y_2) = 2^n - y_2$
- Common method for representing signed numbers

$$C(10011_2) = 2^5 - 19 = 13$$
$$= 01101_2$$

$$C(0110_2) = 10000_2$$
$$- 00110_2$$
$$= \ \ 1010_2$$

Ones' Complement

- b's complement of an n-digit number y_b in base b:

  $$C(y_b) = b^n - y_b$$

- Ones' complement of an n-bit number y_2:

  $$D(y_2) = 2^n - 1 - y_2$$

- In binary ($b = 2$), C is the two's complement and D is the ones' complement

- Add 1 to obtain the b's complement:

  $$C(y_b) = D(y_b) + 1$$

$$\begin{aligned} D(0110_2) &= 1111_2 \\ &\quad - 0110_2 \\ &= 1001_2 \end{aligned}$$

Invert all bits to obtain the ones' complement.

$$\begin{aligned} C(0110_2) &= 1001_2 \\ &\quad + 0001_2 \\ &= 1010_2 \end{aligned}$$

Add 1 to obtain the two's complement; keep the original bit width.

Subtraction through Addition

- General base b: use the b's complement to perform subtraction by an addition:

$$\begin{aligned} z_b &= x_b - y_b \\ &= x_b + b^n - y_b - b^n \\ &= x_b + C(y_b) - b^n \end{aligned}$$

- Assume x_b and y_b have both n digits and $x_b \geq y_b > 0$, then:

$$b^n > x_b - y_b \geq 0 \Longrightarrow x_b + C(y_b) \in [b^n, 2b^n)$$

- A number in $[b^n, 2b^n)$ has a leading one at position $n + 1$, dropping it is equivalent to subtracting b^n
- In summary, do the following:

1. Find b's complement of subtrahend
2. Add complement to minuend
3. Discard the carry beyond the original n digits

Subtraction through Addition

- Task: compute $z_2 = x_2 - y_2$
- $C(y_2) = 2^n - y_2$
- In summary, do the following:
    1. Find the two's complement of the subtrahend
    2. Add it to the minuend
    3. Discard the carry beyond the original n bits

$$x_2 = 110110_2 = 54_{10}$$
$$y_2 = 101011_2 = 43_{10}$$

1. $$C(y_2) = 010100_2 + 1$$
$$= 010101_2$$

2. $$x_2 + C(y_2) = \quad 110110_2$$
$$+ \quad 010101_2$$
$$= 1001011_2$$

3. $x_2 - y_2 = 001011_2 = 11_{10}$

Characters and Strings

Characters

- Data is stored as a sequence of bits, how to store textual information?
- Follow approach of numerical data: interpret bits according to a scheme / standard
- American Standard Code for Information Interchange (ASCII):
  - Designed for US-English, not sufficient for all use cases
  - ASCII defines 128 codes using seven bits; it is commonly stored in an eight-bit byte
  - Split into non-printing and printing characters
  - Extensions exist, typically compatible to ASCII

ASCII control characters

Hex	Abbr.	Meaning
00	NUL	Null
01	SOH	Start of heading
02	STX	Start of text
03	ETX	End of text
04	EOT	End of transmission
05	ENQ	Enquiry
06	ACK	Acknowledge
07	BEL	Bell
08	BS	Backspace
09	HT	Horizontal tab
0A	LF	Line feed
0B	VT	Vertical tab
0C	FF	Form feed
0D	CR	Carriage return
0E	SO	Shift out
0F	SI	Shift in

Hex	Abbr.	Meaning
10	DLE	Data link escape
11	DC1	Device control 1
12	DC2	Device control 2
13	DC3	Device control 3
14	DC4	Device control 4
15	NAK	Negative acknowledge
16	SYN	Synchronous idle
17	ETB	End of transmission block
18	CAN	Cancel
19	EM	End of medium
1A	SUB	Substitute
1B	ESC	Escape
1C	FS	File separator
1D	GS	Group separator
1E	RS	Record separator
1F	US	Unit separator
7F	DEL	Delete

Source: RFC 20, ASCII format for Network Interchange.

Characters

- Data is stored as a sequence of bits, how to store textual information?
- Follow approach of numerical data: interpret bits according to a scheme / standard
- American Standard Code for Information Interchange (ASCII):
  - Designed for US-English, not sufficient for all use cases
  - ASCII defines 128 codes using seven bits; it is commonly stored in an eight-bit byte
  - Split into non-printing and printing characters
  - Extensions exist, typically compatible to ASCII

ASCII

High bits: Hex		0	1	2	3	4	5	6	7
High bits: Binary		000	001	010	011	100	101	110	111
Low bits Hex	Low bits Binary	Characters							
0	0000	NUL	DLE	SP	0	@	P	`	p
1	0001	SOH	DC1	!	1	A	Q	a	q
2	0010	STX	DC2	"	2	B	R	b	r
3	0011	ETX	DC3	#	3	C	S	c	s
4	0100	EOT	DC4	$	4	D	T	d	t
5	0101	ENQ	NAK	%	5	E	U	e	u
6	0110	ACK	SYN	&	6	F	V	f	v
7	0111	BEL	ETB	'	7	G	W	g	w
8	1000	BS	CAN	(	8	H	X	h	x
9	1001	HT	EM	)	9	I	Y	i	y
A	1010	LF	SUB	*	:	J	Z	j	z
B	1011	VT	ESC	+	;	K	[	k	{
C	1100	FF	FS	,	<	L	\	l	|
D	1101	CR	GS	-	=	M	]	m	}
E	1110	SO	RS	.	>	N	^	n	~
F	1111	SI	US	/	?	O	_	o	DEL

Source: RFC 20, ASCII format for Network Interchange.

Strings

- String: sequence of characters
- Each ASCII code can be stored in one byte, with the most significant bit set to zero
- C-style strings end with a zero byte (`0x00`)

ASCII

High bits: Hex		0	1	2	3	4	5	6	7
High bits: Binary		000	001	010	011	100	101	110	111
Low bits Hex	Low bits Binary	Characters							
0	0000	NUL	DLE	SP	0	@	P	`	p
1	0001	SOH	DC1	!	1	A	Q	a	q
2	0010	STX	DC2	"	2	B	R	b	r
3	0011	ETX	DC3	#	3	C	S	c	s
4	0100	EOT	DC4	$	4	D	T	d	t
5	0101	ENQ	NAK	%	5	E	U	e	u
6	0110	ACK	SYN	&	6	F	V	f	v
7	0111	BEL	ETB	'	7	G	W	g	w
8	1000	BS	CAN	(	8	H	X	h	x
9	1001	HT	EM	)	9	I	Y	i	y
A	1010	LF	SUB	*	:	J	Z	j	z
B	1011	VT	ESC	+	;	K	[	k	{
C	1100	FF	FS	,	<	L	\	l	|
D	1101	CR	GS	-	=	M	]	m	}
E	1110	SO	RS	.	>	N	^	n	~
F	1111	SI	US	/	?	O	_	o	DEL

Source: RFC 20, ASCII format for Network Interchange.

Example: derive binary and hex encoding of null-terminated string “Hello World”

Char	Binary	Hex
H	01001000	48
e	01100101	65
l	01101100	6c
l	01101100	6c
o	01101111	6f
SP	00100000	20
W	01010111	57
o	01101111	6f
r	01110010	72
l	01101100	6c
d	01100100	64
NUL	00000000	00

ASCII

High bits: Hex		0	1	2	3	4	5	6	7
High bits: Binary		000	001	010	011	100	101	110	111
Low bits Hex	Low bits Binary	Characters							
0	0000	NUL	DLE	SP	0	@	P	`	p
1	0001	SOH	DC1	!	1	A	Q	a	q
2	0010	STX	DC2	"	2	B	R	b	r
3	0011	ETX	DC3	#	3	C	S	c	s
4	0100	EOT	DC4	$	4	D	T	d	t
5	0101	ENQ	NAK	%	5	E	U	e	u
6	0110	ACK	SYN	&	6	F	V	f	v
7	0111	BEL	ETB	'	7	G	W	g	w
8	1000	BS	CAN	(	8	H	X	h	x
9	1001	HT	EM	)	9	I	Y	i	y
A	1010	LF	SUB	*	:	J	Z	j	z
B	1011	VT	ESC	+	;	K	[	k	{
C	1100	FF	FS	,	<	L	\	l	|
D	1101	CR	GS	-	=	M	]	m	}
E	1110	SO	RS	.	>	N	^	n	~
F	1111	SI	US	/	?	O	_	o	DEL

Source: RFC 20, ASCII format for Network Interchange.

Summary and Outlook

- Different data types exist: integers, strings, floating-point numbers, etc.
- Machines store data as binary
- The encoding determines how a bit pattern is interpreted
- Machine code (programs) is nothing other than a sequence of bits:
  - We need an interpretation as well
  - Later in this course

